

Relatório Técnico

AT de Visão Computacional

Exercícios 1 a 4

Calibração de câmera e realidade aumentada · Classificação com OpenCV DNN e pipeline integrado · Detecção YOLO/SSD com rastreamento · Segmentação semântica e relatório integrativo

Autor: Rodrigo Vianna (Dev-RMV)

Data de execução de referência: 23 de setembro de 2026

Ambiente: Windows 11 Pro, CPU Intel Core i9-14900KF (sem GPU), Python 3.13.5, OpenCV 4.14.0 (opencv-contrib-python), TensorFlow 2.21.0, Keras 3.15.1, NumPy 2.5.3

Repositório (GitHub): https://github.com/Dev-RMV/AT_Visao_Computacional

Cópia local usada nas execuções: C:\AT_visao_computacional_v2 (pastas ex1/ a ex4/, prints/, criar_ambiente.py)

Conteúdo: (1) comandos de instalação do ambiente e de execução de cada script; (2) justificativas objetivas das decisões adotadas em cada exercício; (3) evidências de execução: capturas de terminal, capturas das janelas OpenCV, gráficos das métricas medidas, tabelas comparativas e referência ao vídeo de demonstração do Exercício 3b.

Sumário

1. Escopo e organização da entrega	3
2. Ambiente: instalação e execução dos scripts	4
2.1 Requisitos	4
2.2 Instalação do ambiente (um único venv para os quatro exercícios)	4
2.3 Execução de cada script	4
3. Exercício 1: calibração de câmera e realidade aumentada	7
3.1 Decisões adotadas e justificativas	7
3.2 Resultados medidos	7
3.3 Evidências	8
4. Exercício 2: classificação com OpenCV DNN vs Keras e pipeline integrado	10
4.1 Decisões adotadas e justificativas	10
4.2 Resultados medidos	10
4.3 Evidências	11
5. Exercício 3: detecção com YOLO e SSD e rastreamento por IoU	15
5.1 Decisões adotadas e justificativas	15
5.2 Resultados medidos (795 frames de vtest.avi, CPU, NMS 0,4)	15
5.3 Evidências	16
5.4 Síntese da discussão ética (relatorio_drone_etica.md)	17
6. Exercício 4: segmentação semântica e relatório integrativo	18
6.1 Decisões adotadas e justificativas	18
6.2 Resultados medidos	18
6.3 Evidências do Item A	19
6.4 Síntese do relatório integrativo (Item B)	20
7. Limitações declaradas e o que não foi medido	23
8. Índice das evidências de execução	23

1. Escopo e organização da entrega

O código, as entradas, os modelos, as saídas e este relatório estão no repositório público github.com/Dev-RMV/AT_Visao_Computacional.

Este relatório documenta os quatro exercícios do AT de Visão Computacional. Cada exercício está em uma pasta própria com o mesmo padrão: **main.py** (implementação completa dos itens A e B), scripts de conveniência **exercicio0Na.py** e **exercicio0Nb.py**, **enunciado.txt**, **requirements.txt**, as pastas **entradas/**, **modelos/** e **saidas/**, e um relatório em Markdown gerado ou escrito para o exercício. A pasta **prints/** guarda as capturas de tela feitas durante as execuções no PyCharm, e o arquivo **verificacao.md** registra a auditoria feita sobre o código antes da entrega.

```
AT_visao_computacional_v2/
├── criar_ambiente.py          # cria .venv e instala os requirements de ex1..ex4
├── todos_os_enunciados.txt    # enunciados dos 4 exercícios
├── verificacao.md            # auditoria do código contra os enunciados e revalidação
├── prints/                   # 8 capturas de tela (terminal, janelas OpenCV, notebook)
├── ex1/
│   ├── main.py, exercicio01a.py, exercicio01b.py, relatorio_ex1.md
│   ├── entradas/ 20 imagens sint_XX.png, video_sintetico.avi, verdade.npz
│   └── saidas/   calibracao.npz, painel_undistort.png, video_aumentado.avi,
│                   poses_estimadas.csv, frame_cubo_*.png
├── ex2/
│   ├── main.py, exercicio02a.py, exercicio02b.py, exercicio02b.ipynb, relatorio_ex2.md
│   ├── entradas/ 10 imagens Imagenette, rotulos.csv, bola.mp4, calibracao.npz
│   ├── modelos/  mobilenetv2_imagenet.tflite, imagenet_class_index.json
│   └── saidas/   top3/, frame_pipeline_integrado.png
├── ex3/
│   ├── main.py, relatorio_drone_etica.md
│   ├── entradas/ vtest.avi, video_sintetico.avi
│   ├── modelos/  yolov4-tiny.cfg/.weights, ssd_mobilenet_v2_coco.pb/.pbtxt, coco.names
│   └── saidas/   item_a_yolo.avi, item_a_ssd.avi, item_b_rastreamento.avi, frames .png
├── ex4/
│   ├── main.py, exercicio04a.py, exercicio04b.py, relatorio_integrativo.md
│   ├── entradas/ img01..img06.png
│   ├── modelos/  deeplabv3_mnv2_ade20k.pb, ade20k_objectInfo150.csv
│   └── saidas/   mapa_*, overlay_*, painel_comparativo_*, porcentagens_classes.csv,
│                   relatorio_integrativo.md
```

Sobre as evidências pedidas no enunciado da entrega:

- **Prints de terminal e capturas das janelas OpenCV:** as oito capturas da pasta prints/ estão reproduzidas nas seções de cada exercício, junto com as imagens gravadas pelos próprios scripts em saidas/.
- **Gráficos de treino:** nenhum modelo foi treinado neste AT. Todos os modelos são pré-treinados (MobileNetV2 no ImageNet, YOLOv4-tiny e SSD MobileNetV2 no COCO, DeepLabV3-MobileNetV2 no ADE20K), portanto não existem curvas de treino. Os gráficos apresentados são das métricas medidas nas execuções (latência, FPS, memória, erro de reprojeção, jitter, área por classe).
- **Tabelas comparativas:** reproduzidas com os números da execução de referência de 23/09/2026 (comando **python main.py --sem-janela** em cada pasta).
- **Vídeo de demonstração do Exercício 3b:** o arquivo **ex3/saidas/item_b_rastreamento.avi** (795 frames, XVID, 22 MB) acompanha a entrega; dois frames dele estão reproduzidos na seção 5.

As capturas de tela foram feitas em execuções **com as janelas do OpenCV abertas**, que são um pouco mais lentas que a execução de referência sem janela. Por isso alguns valores instantâneos nos prints (por exemplo 34,3 FPS no Ex1 B ou 39,82 FPS no YOLO) diferem ligeiramente dos valores das tabelas, medidos com **--sem-janela**. As ordens de grandeza e as conclusões são as mesmas.

2. Ambiente: instalação e execução dos scripts

2.1 Requisitos

- Windows 11 (os scripts também rodam em Linux; os caminhos de ativação do venv mudam).
- **Python 3.13.** O TensorFlow 2.21 só publica wheel até o 3.13, e o criar_ambiente.py recusa versões abaixo de 3.10.
- CPU com AVX2. Não há uso de GPU: no Windows nativo o TensorFlow executa apenas em CPU, e o OpenCV DNN usa o backend CPU padrão.
- Cerca de 3 GB livres em disco para o venv (TensorFlow) e 65 MB para os vídeos gerados pelo Exercício 3.

2.2 Instalação do ambiente (um único venv para os quatro exercícios)

O código é clonado do GitHub (github.com/Dev-RMV/AT_Visao_Computacional). Os quatro arquivos requirements.txt são idênticos. O script criar_ambiente.py cria o venv com o interpretador que o executa, atualiza o pip e instala os requirements de cada exercício; rodar de novo instala só o que faltar.

```
# 1. obter o código (PowerShell ou cmd)
git clone https://github.com/Dev-RMV/AT_Visao_Computacional.git
cd AT_Visao_Computacional

# 2. na raiz do projeto
py -3.13 --version          # confirma o interpretador (3.13.x)
py -3.13 criar_ambiente.py  # cria .venv\ e instala ex1..ex4/requirements.txt
.venv\Scripts\activate     # ativa o ambiente (Linux/macOS: source .venv/bin/activate)
python -m pip check         # esperado: "No broken requirements found."
```

Alternativa manual, equivalente ao script:

```
py -3.13 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r ex1\requirements.txt
```

Pacotes instalados e motivo de cada um:

Pacote	Versão	Uso no AT
opencv-contrib-python	4.14.0.94	Calibração, solvePnP, projectPoints, undistort, HSV, ORB, HOG, cv2.dnn (TFLite, Darknet, TensorFlow), VideoCapture/VideoWriter, janelas
numpy	2.5.3	Arrays de imagens, pontos 3D/2D, .npz da calibração
tensorflow	2.21.0	MobileNetV2 nativa no Keras (Ex2 A) e DeepLabV3 em grafo congelado via tf.compat.v1 (Ex4 A)
keras	3.15.1	keras.applications.MobileNetV2 com pesos ImageNet
psutil	7.2.2	Memória RSS do processo na tabela DNN vs Keras
nbformat / nbclient / ipykernel	≥ 5.10 / ≥ 0.10 / ≥ 6.29	Geração e execução real do notebook exercicio02b.ipynb com o kernel do venv
matplotlib	3.11.2	Exibição da figura dentro do notebook do Ex2 B

Regra registrada nos requirements: nunca instalar opencv-python e opencv-contrib-python juntos, porque os dois escrevem no mesmo módulo cv2 e o resultado é imprevisível.

2.3 Execução de cada script

Todos os comandos abaixo pressupõem o venv ativado e o terminal **dentro da pasta do exercício** (os scripts resolvem entradas/, modelos/ e saidas/ em relação ao próprio arquivo). Sem argumentos, cada main.py executa os itens A e B em sequência e abre as janelas do OpenCV; a flag --sem-janela apenas grava as saídas.

Exercício 1: calibração e realidade aumentada

```
cd ex1
python main.py # Itens A e B com janelas (painel e feed com o cubo)
python main.py --sem-janela # execução de referência (A + B em ~3 s)
python main.py --item a # só a calibração (K, distorção, erro, painel)
python main.py --item b --max-frames 20 # só a RA, limitada a 20 frames
python main.py --item b --video entradas\video_sintetico.avi # outro vídeo do tabuleiro
python exercicio01a.py --sem-janela # atalho para o Item A
python exercicio01b.py --max-frames 20 # atalho para o Item B (--sem-janela, --video)
```

Exercício 2: OpenCV DNN vs Keras e pipeline integrado

```
cd ex2
python main.py --sem-janela # Itens A e B; gera e executa exercicio02b.ipynb (~20 s)
python main.py --item a --loops 20 # só o benchmark (20 repetições por imagem; padrão)
python main.py --item b --frame 180 # só o pipeline, no frame 180 de bola.mp4 (padrão)
python exercicio02a.py --loops 3 # atalho rápido para o Item A
python exercicio02b.py --frame 180 # atalho para o Item B (gera e executa o notebook)
jupyter notebook exercicio02b.ipynb # opcional: abrir o notebook já executado
```

Exercício 3: YOLOv4-tiny, SSD MobileNetV2 e rastreamento

```
cd ex3
python main.py --sem-janela # 795 frames de vtest.avi: YOLO, SSD e rastreamento (~62 s)
python main.py # o mesmo, com as três janelas ao vivo (mais lento)
python main.py --max-frames 150 # teste rápido com 150 frames
python main.py --video entradas\meu.avi # outro vídeo
python main.py --sintetico # gera e usa o vídeo sintético (recortes reais de vtest.avi)
```

Exercício 4: segmentação semântica e relatório integrativo

```
cd ex4
python main.py --sem-janela # Item A (6 cenas) + Item B (gera e valida o relatório) (~3 s)
python main.py --item a # só a segmentação, com os painéis em janela
python main.py --item b # só o relatório (executa A sem janela para medir)
python exercicio04a.py --sem-janela # atalho para o Item A
python exercicio04b.py # atalho para o Item B
```

Flags comuns e seus efeitos:

Flag	Scripts	Efeito
--item {a,b,ambos}	ex1, ex2, ex4	Escolhe o item; o padrão é ambos
--sem-janela	todos	Não abre janelas do OpenCV; grava tudo em saidas/. Usada na execução de referência
--max-frames N	ex1, ex3	Limita os frames processados (0 = todos)
--video CAMINHO	ex1, ex3	Troca o vídeo de entrada
--loops N	ex2	Repetições por imagem no benchmark (padrão 20)
--frame N	ex2	Frame de bola.mp4 usado no pipeline (padrão 180)

Flag	Scripts	Efeito
--sintetico	ex3	Força o vídeo sintético mesmo com vtest.avi presente

Saída esperada de cada execução (terminal e arquivos):

Exercício	Terminal	Arquivos em saidas/
1	K, 5 coeficientes, erro por imagem e médio, classificação; rvec/tvec a cada frame; resumo de estabilidade	calibracao.npz, painel_undistort.png, video_aumentado.avi, poses_estimadas.csv, frame_cubo_{inicial,meio,final}.png
2	Top-1 por imagem, tabela DNN vs Keras, comentário de backend; tempo por etapa do pipeline	top3/ (10 imagens), frame_pipeline_integrado.png; exercicio02b.ipynb executado na pasta do exercício
3	Parâmetros/tamanho dos modelos, latência e FPS por modelo, tabela comparativa, conclusão; contagem e ID switches	item_a_yolo.avi, item_a_ssd.avi, item_b_rastreamento.avi e frames .png; relatorio_drone_etica.md na pasta do exercício
4	Porcentagem por classe em cada cena, latências, discussão; validação das 5 seções e contagem de palavras	mapa_*, overlay_*, painel_comparativo_* (6 cenas), porcentagens_classes.csv, relatorio_integrativo.md

3. Exercício 1: calibração de câmera e realidade aumentada

3.1 Decisões adotadas e justificativas

- **Câmera virtual com gabarito conhecido.** As 20 imagens do tabuleiro (9×6 cantos internos, quadrado de 25 mm) e o vídeo foram sintetizados por uma câmera virtual cujos parâmetros verdadeiros estão em entradas/verdade.npz. Isso permite medir o erro real da calibração (K e distorção contra o valor verdadeiro), o que é impossível com uma câmera física. O enunciado pede ao menos 15 imagens; foram usadas 20 para condicionar melhor a otimização.
- **Refinamento subpixel antes da calibração.** `cv2.findChessboardCorners` seguido de `cv2.cornerSubPix` (critério EPS 0,001). Cantos refinados reduzem o erro de reprojeção e são o que torna o cubo do Item B estável.
- **Modelo de distorção com 5 coeficientes** (k_1 , k_2 , p_1 , p_2 , k_3) sem flags de fixação, como o enunciado exige. O código comenta o significado físico de cada parâmetro: f_x e f_y (distância focal × densidade de pixels, definem escala métrica e FOV), c_x e c_y (ponto principal), k_1 .. k_3 (distorção radial; $k_1 < 0$ é barril), p_1 e p_2 (descentramento).
- **Limiares de aceitação codificados e aplicados ao valor medido.** Erro médio $< 0,5$ px: excelente (SLAM, RA, manipulação); $0,5$ a $1,0$ px: aceitável (navegação de robôs móveis); $> 1,0$ px: inaceitável. A classificação impressa no terminal é derivada do erro, não um texto fixo.
- **Pose com continuidade temporal.** `cv2.solvePnP` em modo ITERATIVE com `useExtrinsicGuess` alimentado pela pose do frame anterior e fallback para IPPE (planar) no primeiro frame ou em falha; verificação da orientação do canto de origem para impedir a inversão de 180° do tabuleiro, que faria o cubo virar.
- **Cubo com aresta de 1 quadrado na origem** (0, 0, 0), projetado com `cv2.projectPoints`, 6 faces com cores distintas e preenchimento semitransparente ordenado por profundidade, além dos eixos X/Y/Z. Legenda de cores no próprio frame.
- **Estabilidade medida, não afirmada.** O jitter é a segunda diferença temporal dos vértices projetados (px/frame²). O veredito ESTÁVEL só é impresso se a detecção ocorrer em ≥ 95 % dos frames e o jitter médio for < 1 px/frame².
- **Vídeo gravado em vez de webcam.** O enunciado admite as duas formas; o vídeo (150 frames, 1280×720) torna o resultado reproduzível. O terminal imprime `rvec` e `tvec` a cada frame e o CSV guarda a pose completa.

3.2 Resultados medidos

Parâmetro	Valor verdadeiro	Calibrado	Erro absoluto	Erro relativo
f_x (px)	1000,0000	1000,3806	+0,3806	+0,038 %
f_y (px)	1010,0000	1010,4898	+0,4898	+0,048 %
c_x (px)	652,0000	652,0135	+0,0135	+0,002 %
c_y (px)	351,0000	351,2732	+0,2732	+0,078 %
k_1	-0,30000	-0,30246	-0,00246	–
k_2	+0,10000	+0,11252	+0,01252	–
p_1	+0,00100	+0,00096	-0,00004	–
p_2	-0,00060	-0,00062	-0,00002	–
k_3	-0,01500	-0,03407	-0,01907	–

Métrica (Item A e B)	Valor
Erro médio de reprojeção (20 imagens)	0,0482 px (classificação impressa: EXCELENTE, < 0,5 px)
Erro RMS global	0,0491 px
Frames com pose e cubo	150 / 150 (taxa de detecção 100 %)
Taxa de processamento	56,8 FPS sem janela (34,3 FPS com janela, no print)
Jitter médio / máximo	0,1975 / 0,3102 px/frame ²
Veredito impresso	ESTÁVEL E ADERIDO AO TABULEIRO (condicional: detecção ≥ 95 % e jitter < 1)

3.3 Evidências

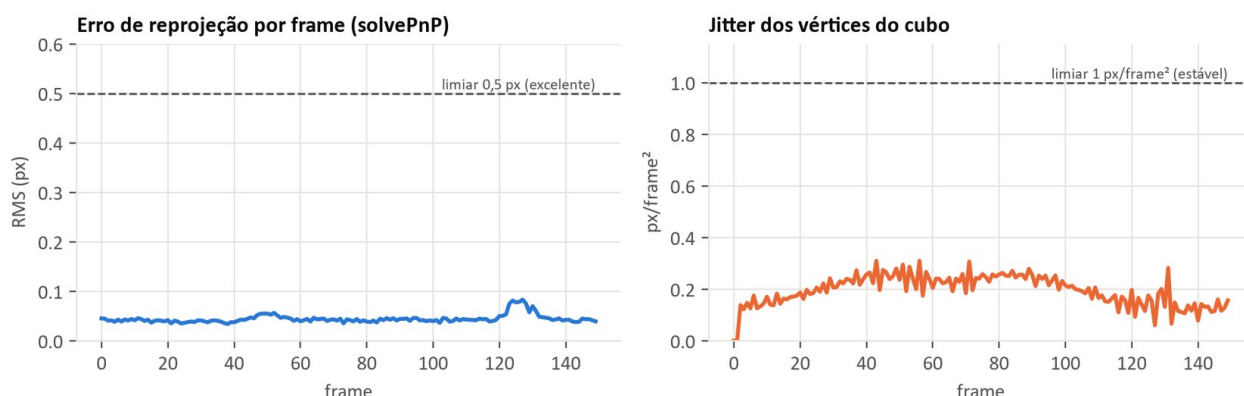


Figura 3.1 – Gráficos a partir de `saidas/poses_estimadas.csv`: erro de reprojeção da pose por frame e jitter dos vértices do cubo, ambos muito abaixo dos limiares codificados.



Figura 3.2 – Item A: painel gravado por `main.py` com a imagem original (grade curvada pela distorção de barril, $k1 = -0,30$) e a corrigida com `cv2.undistort` (linhas retas).

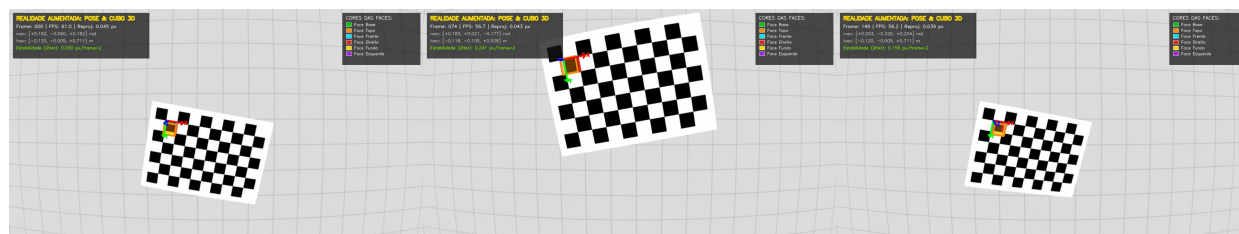


Figura 3.3 – Item B: cubo 3D projetado sobre a origem do tabuleiro no primeiro, no central e no último frame do vídeo (`saidas/frame_cubo_*.png`), com HUD de `rvec`, `tvec` e `jitter`.

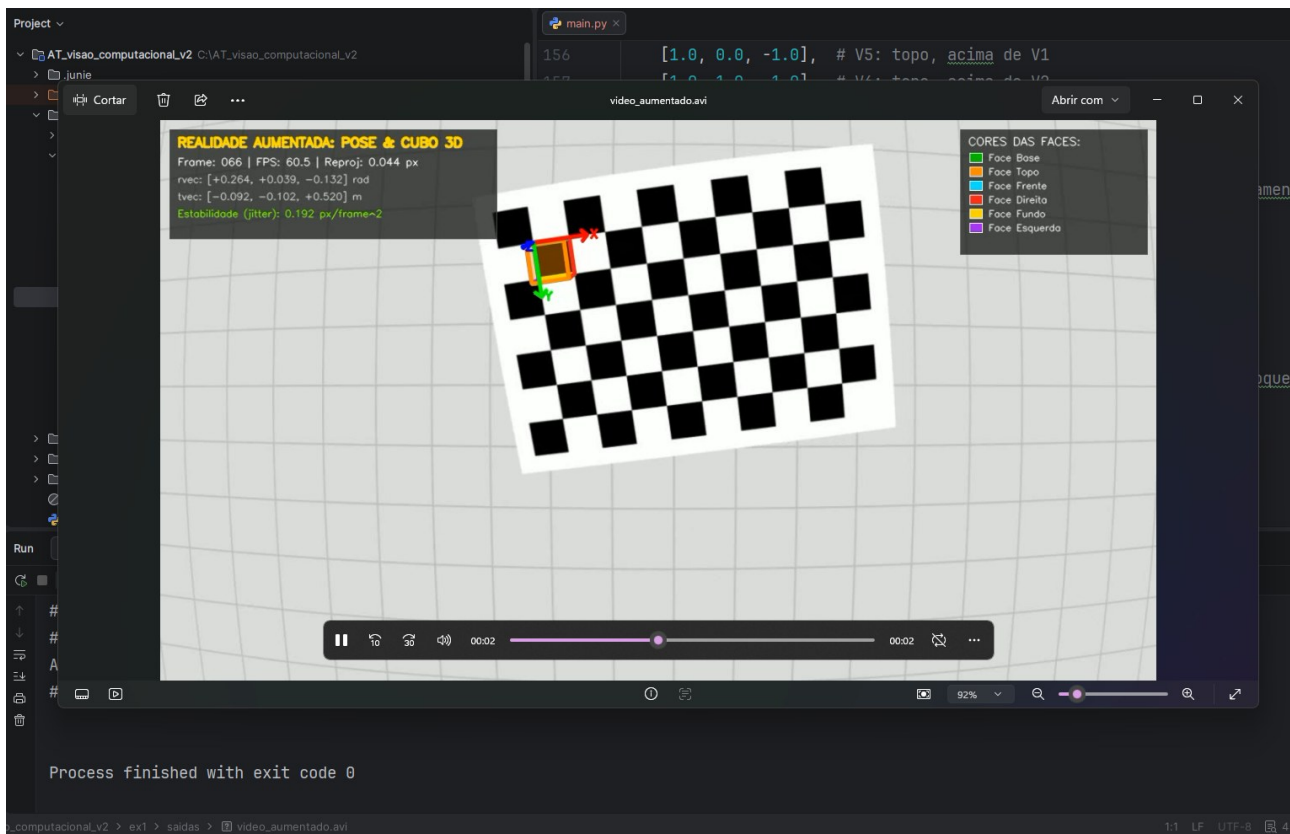


Figura 3.4 – Print: reprodução de saídas/video_aumentado.avi (frame 66) com o cubo, os eixos, a telemetria rvec/tvec e a legenda das faces; abaixo, o terminal com "Process finished with exit code 0".

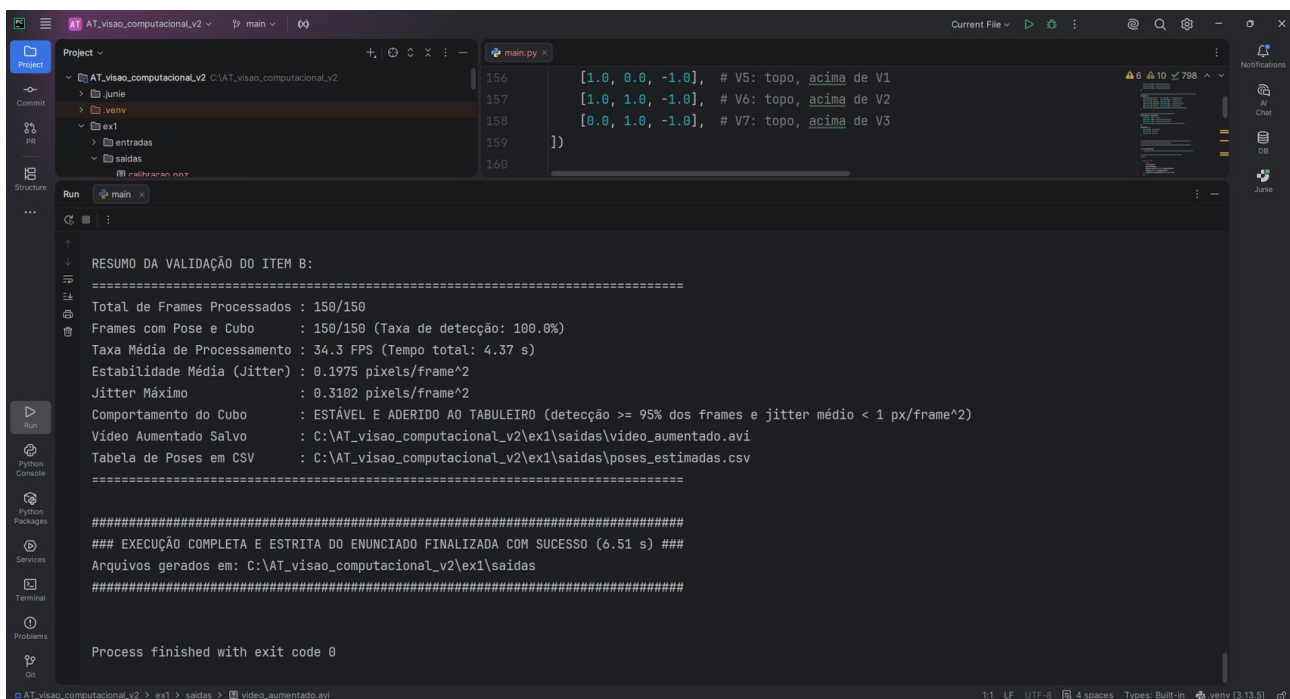


Figura 3.5 – Print do terminal: resumo da validação do Item B (150/150 frames, jitter médio 0,1975 px/frame², veredito condicional ESTÁVEL) e caminho dos arquivos gerados.

4. Exercício 2: classificação com OpenCV DNN vs Keras e pipeline integrado

4.1 Decisões adotadas e justificativas

- **MobileNetV2 em TensorFlow/TFLite**, entre as opções do enunciado, porque a mesma rede existe em keras.applications com os mesmos pesos ImageNet: a comparação "mesma rede nos dois backends" fica justa. O OpenCV lê o arquivo com `cv2.dnn.readNetFromTFLite`; o Keras carrega `MobileNetV2(weights='imagenet')`.
- **Blob idêntico ao pré-processamento do Keras**: `cv2.dnn.blobFromImage` com escala 1/127,5, tamanho 224×224, média 127,5 e `swapRB=True`, que leva os pixels ao intervalo [-1, 1] exatamente como `preprocess_input`. As previsões top-1 coincidiram nas 10 imagens, o que confirma a equivalência.
- **10 imagens de 10 classes distintas (Imagenette) com gabarito em rotulos.csv**. A acurácia top-1 é medida contra o gabarito, com o mesmo critério nos dois backends (igualdade após normalização; sinônimos separados por "|", como `cassette_player|tape_player`). Imagem sem gabarito sai da conta com aviso.
- **Benchmark honesto**: 5 inferências de aquecimento e 20 repetições por imagem com batch 1; o Keras roda em subprocesso separado para que o RSS do runtime TensorFlow seja medido isoladamente; o RSS é lido com `psutil` depois do aquecimento nos dois casos. Se o subprocesso falhar, o script levanta erro em vez de imprimir números de fallback.
- **Comentário de backend com os valores medidos**: o texto impresso no terminal usa a latência e a memória da própria execução, e o código explica quando o OpenCV DNN é preferível (memória, determinismo, ausência de interpretador Python em nós C++) e quando o Keras é a escolha (treino, fine-tuning, prototipagem).
- **Item B: HOG+SVM, não Haar**. O enunciado diz "HOG+SVM ou Haar Cascade"; foi implementado o HOG+SVM de pedestres do OpenCV. Como `bola.mp4` não tem pessoas, 0 detecções é o resultado correto e está documentado.
- **Frame 180 de bola.mp4** como padrão: entre os frames 30 e 75 a bola está cortada pela borda; no 180 ela está inteira, o que dá uma ROI HSV (faixa azul $H\ 90\text{--}130$, $S \geq 60$, $V \geq 40$, com morfologia) e uma classificação com sentido.
- **Notebook executado de verdade**. `exercicio02b.ipynb` é montado com `nbformat` e executado pelo `nbclient` com o kernel do `venv`; as saídas gravadas (tempos e figura) são as produzidas pelas células, não coladas.
- **Limitação declarada**: `entradas/calibracao.npz` é a calibração da câmera virtual do Ex1 e não corresponde à câmera que gravou `bola.mp4`. O `undistort` é executado e cronometrado porque o enunciado exige encadear o Ex1, mas não corrige distorção real desse vídeo.

4.2 Resultados medidos

Métrica	OpenCV DNN (TFLite)	Keras (TensorFlow 2.x)	Razão
Latência média por imagem	5,63 ms	64,69 ms	11,5×
Latência mínima	5,10 ms	60,11 ms	11,8×
Desvio padrão da latência	0,30 ms	4,95 ms	16×
Memória adicional (modelo + aquecimento)	51,7 MB	433,8 MB	8,4×
RSS total do processo	94,2 MB	463,8 MB	4,9×
Acurácia top-1 (10 imagens com gabarito)	90,0 % (9/10)	90,0 % (9/10)	igual

Os dois backends erram a mesma imagem (church classificada como fountain) e acertam as outras nove. Top-3 obtido pelo OpenCV DNN em cada imagem:

Imagem	Gabarito	Top-1	Top-2	Top-3
n01440764_01	tench	tench 91,2 %	reel 2,2 %	barracouta 0,9 %
n02102040_01	English_springer	English_springer 93,7 %	Cardigan 2,0 %	Border_collie 0,7 %
n02979186_01	cassette_player tape_player	tape_player 37,3 %	cassette_player 28,1 %	CD_player 19,2 %
n03000684_01	chain_saw	chain_saw 42,4 %	swing 7,2 %	stretcher 5,9 %
n03028079_01	church	fountain 52,8 % (erro)	spotlight 3,4 %	umbrella 1,7 %
n03394916_01	French_horn	French_horn 93,5 %	trombone 2,2 %	cornet 1,1 %
n03417042_01	garbage_truck	garbage_truck 92,8 %	trailer_truck 4,6 %	moving_van 1,1 %
n03425413_01	gas_pump	gas_pump 86,6 %	cash_machine 10,6 %	pay-phone 0,4 %
n03445777_01	golf_ball	golf_ball 18,0 %	croquet_ball 7,3 %	soccer_ball 3,3 %
n03888257_01	parachute	parachute 98,2 %	airship 0,1 %	balloon 0,1 %

Etapas do pipeline (Item B, frame 180)	Origem	Tempo	Resultado
1. Undistort com a calibração	Ex1	7,29 ms	frame retificado 1280×720
2. Segmentação HSV + morfologia	TP1	1,79 ms	ROI x=425, y=247, 261×258
3. Features ORB (2000 pontos) na ROI	TP2	81,43 ms	62 keypoints
4. Detector HOG+SVM no frame	TP3	77,71 ms	0 pessoas (esperado)
5. MobileNetV2 (OpenCV DNN) na ROI	Ex2 A	13,63 ms	croquet_ball 20,1 % / balloon 16,1 % / vase 11,3 %
Total		181,85 ms	5,5 FPS em CPU

ImageNet não tem a classe "bola azul"; o top-3 mostra as classes mais próximas do vocabulário de 1000 classes, com confiança baixa, que é o comportamento esperado de um classificador fechado fora do seu domínio.

4.3 Evidências

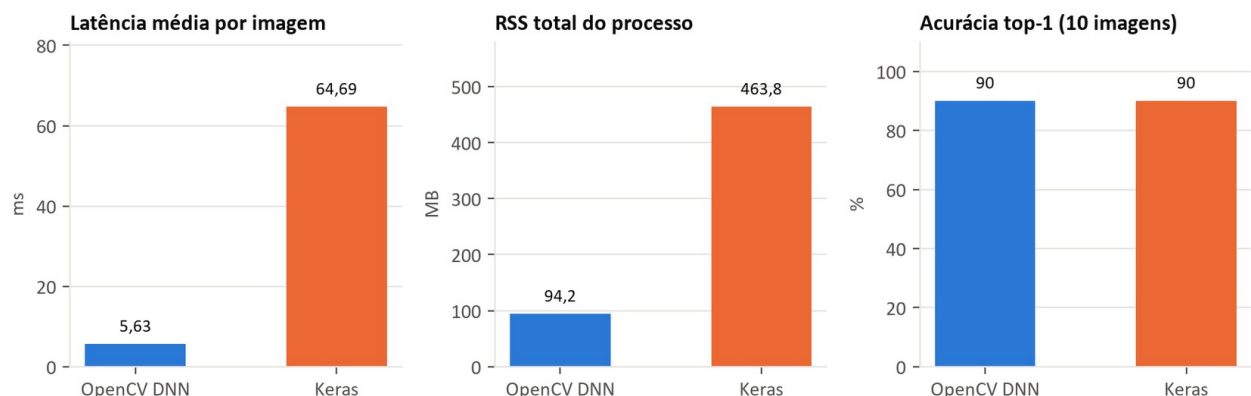


Figura 4.1 – Item A: latência, memória e acurácia dos dois backends (20 repetições por imagem, batch 1).

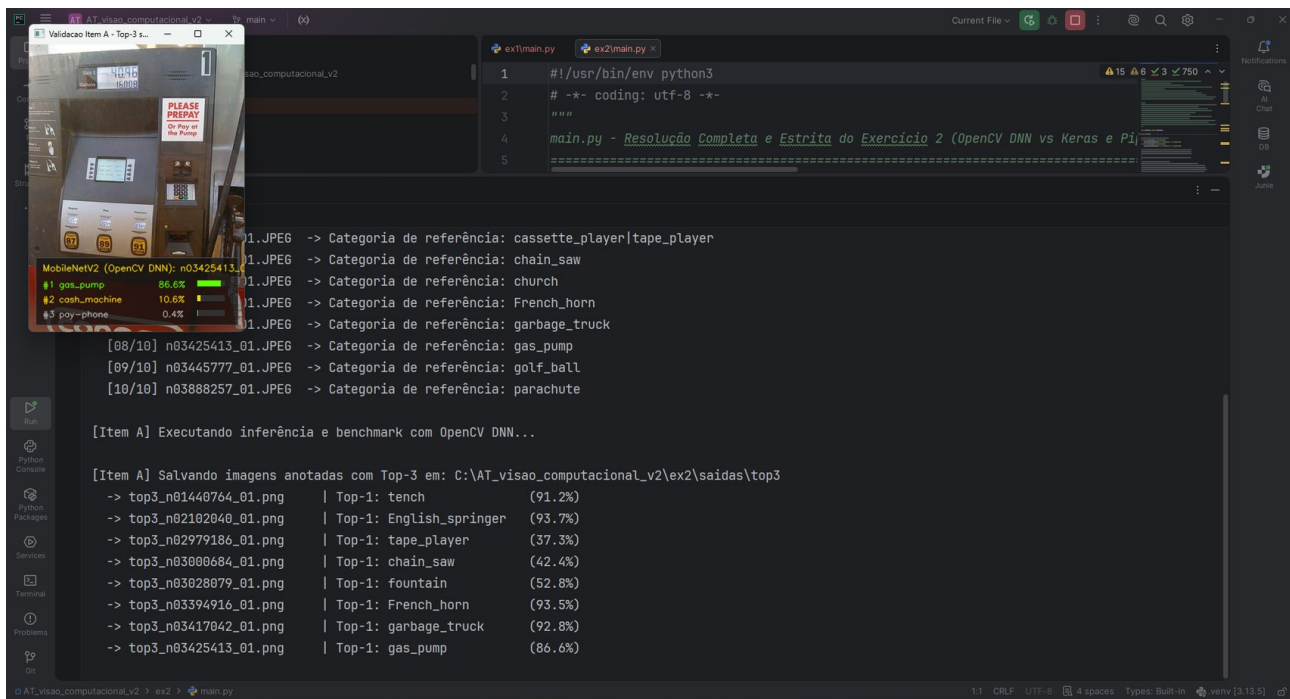


Figura 4.2 – Print: janela OpenCV "Validacao Item A - Top-3" com gas_pump 86,6 % sobreposto à imagem, e o terminal listando a categoria de referência e o top-1 de cada uma das 10 imagens.

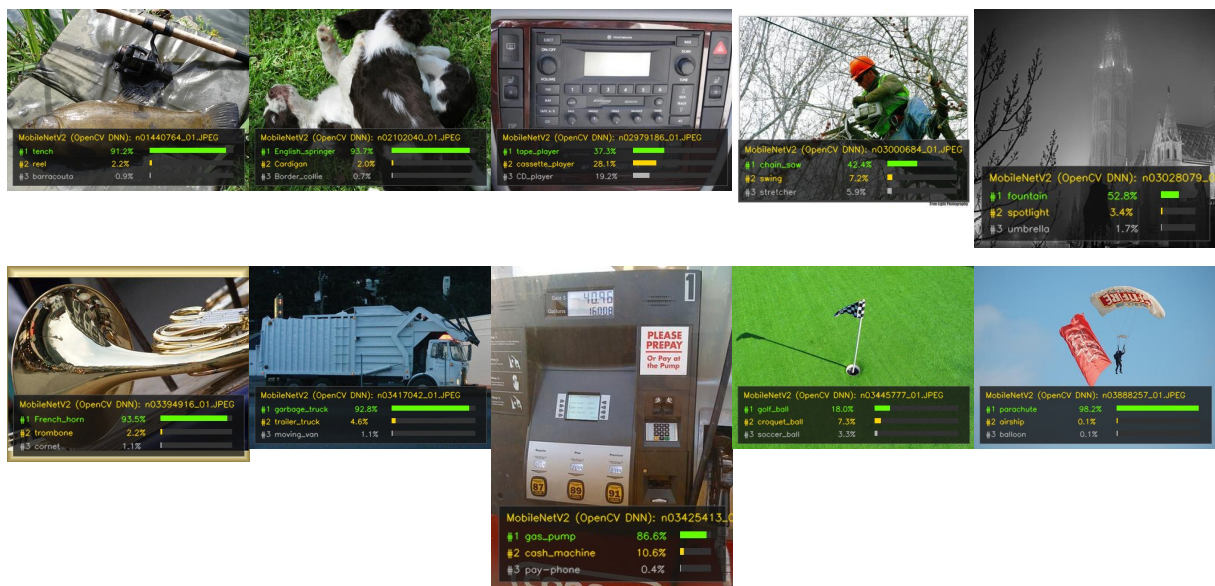


Figura 4.3 – Item A: as 10 imagens anotadas com top-3 e barras de confiança gravadas em saidas/top3/.

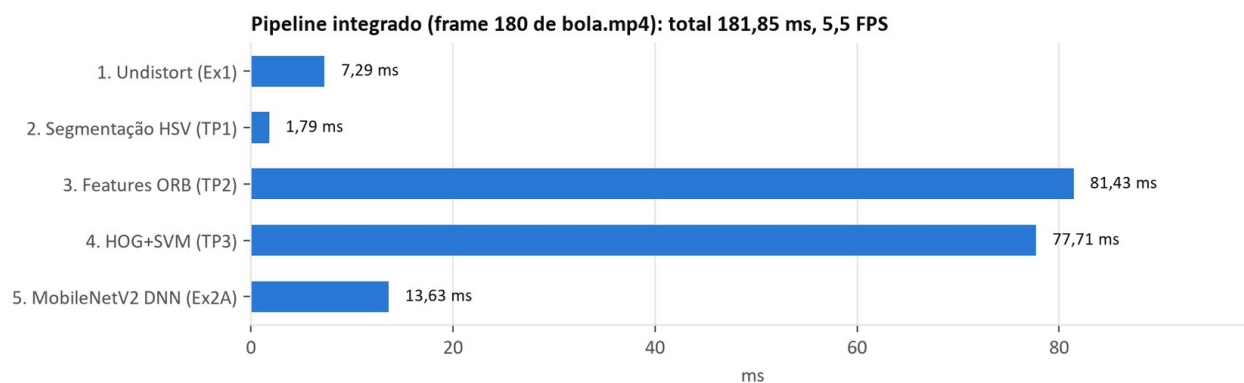


Figura 4.4 – Item B: tempo de cada etapa do pipeline integrado, medido com `time.perf_counter()`.

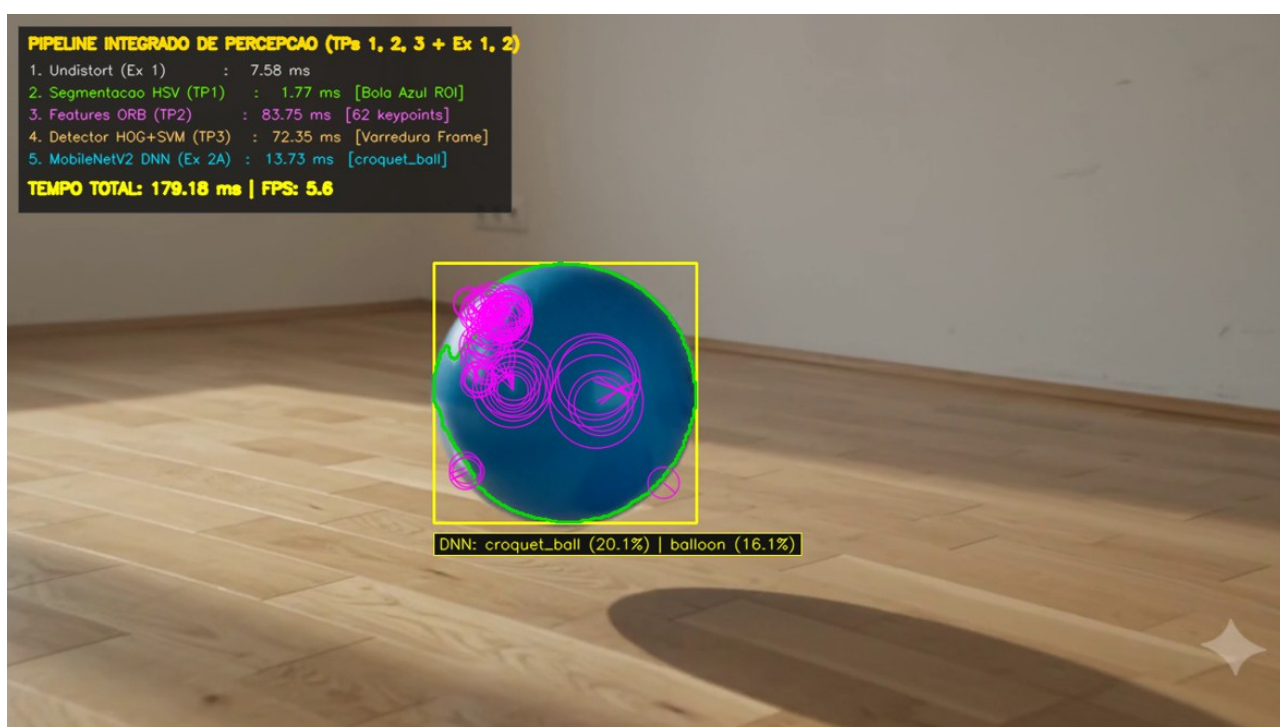


Figura 4.5 – Print da janela do pipeline integrado (`saidas/frame_pipeline_integrado.png`): contorno HSV em verde, keypoints ORB em magenta, caixa da ROI, rótulo do DNN e HUD com o tempo de cada etapa (179,18 ms nesta execução).

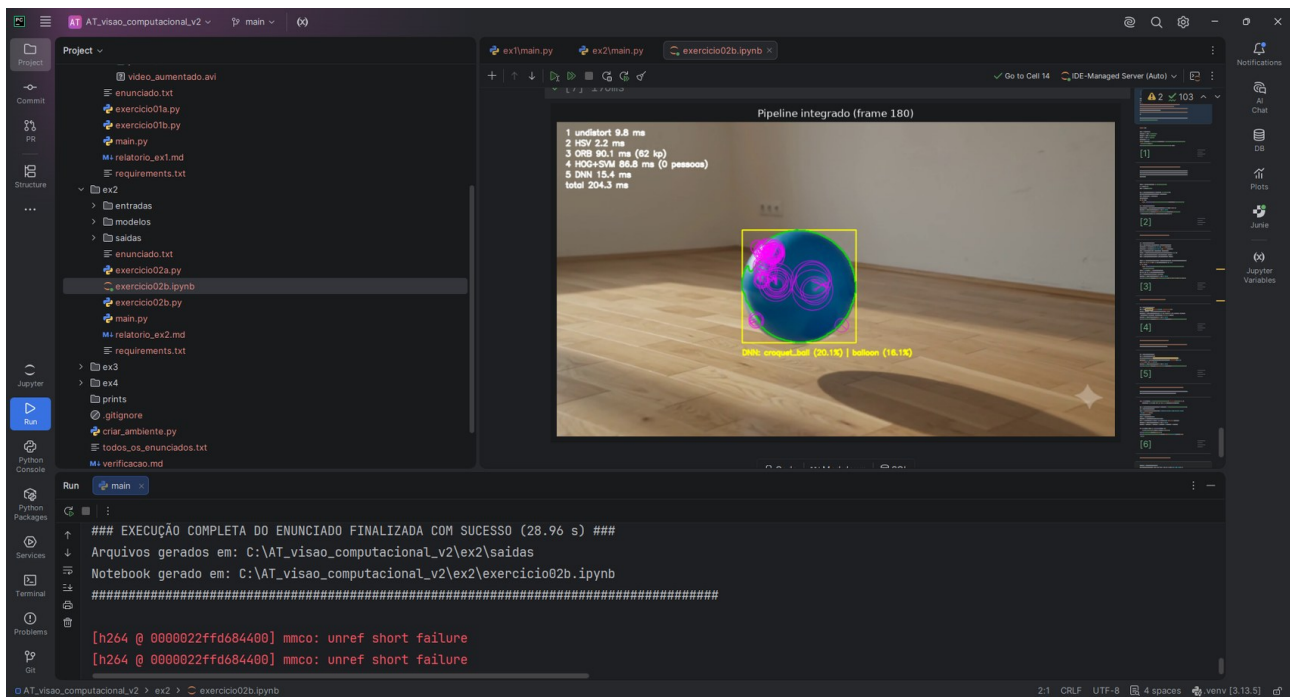


Figura 4.6 – Print: notebook *exercicio02b.ipynb* aberto no PyCharm com a figura gerada pela célula final e o terminal confirmando a execução completa (28,96 s) e a gravação do notebook.

5. Exercício 3: detecção com YOLO e SSD e rastreamento por IoU

5.1 Decisões adotadas e justificativas

- **Os dois detectores no mesmo backend (cv2.dnn).** YOLOv4-tiny em Darknet (cfg/weights, entrada 416×416 com letterboxing) e SSD MobileNetV2 COCO em TensorFlow (pb/pbtxt, entrada 300×300). Mesmo runtime, mesma CPU, mesmo vídeo e mesmo NMS: as diferenças medidas são dos modelos, não da infraestrutura.
- **NMS com threshold 0,4** via cv2.dnn.NMSBoxesBatched (por classe) e confiança mínima 0,5, conforme o enunciado.
- **Número de parâmetros contado no próprio modelo** (soma dos blobs das camadas carregadas) e tamanho em disco lido do arquivo, em vez de valores copiados da literatura.
- **Vídeo real e completo.** vtest.avi (vídeo público do OpenCV com pedestres e veículos, 795 frames, 768×576) processado do início ao fim. O vídeo sintético do repositório serve só como teste de fumaça; ele é montado com recortes reais de pessoas extraídos de vtest.avi pelo próprio YOLO.
- **Conclusão derivada dos dados, não pré-escrita.** A função gerar_conclusao_embarcada aplica um critério explícito: se os dois modelos superam 15 FPS, a taxa não é o gargalo e vence o menor em memória; se só um supera, vence esse; se nenhum, o mais rápido. O trade-off contrário é sempre impresso.
- **Rastreador por IoU com predição linear** (limiar IoU 0,30): track confirmado após 2 detecções (conta como entrada), encerrado após 15 frames sem detecção (conta como saída), trilha dos últimos 30 frames em fila, ID persistente e cor única por ID. Não há filtro de Kalman nem reidentificação por aparência; isso está declarado.
- **Taxa de ID switches apresentada como estimativa heurística.** Sem gabarito anotado para o vídeo, a heurística conta um objeto reidentificado sobre a posição de um track encerrado há poucos frames; trocas entre objetos que se cruzam sem perder a detecção não são capturadas. O docstring, o terminal, o HUD e o relatório dizem isso.
- **Custo do rastreador cronometrado** por frame, para sustentar a afirmação de que a associação é desprezível frente à detecção.
- **Discussão ética em relatório próprio** (relatorio_drone_etica.md): processamento na borda, descarte dos frames, ausência de biometria, risco de efeito inibidor, auditoria de vieses em visada aérea e supervisão humana.

5.2 Resultados medidos (795 frames de vtest.avi, CPU, NMS 0,4)

Modelo	FPS médio	Latência	Parâmetros	Tamanho em disco	Detecções
YOLOv4-tiny (416×416)	40,27	24,83 ms	6,06 M	23,13 MB	5659
SSD MobileNetV2 (300×300)	53,98	18,53 ms	16,88 M	66,46 MB	4942

Conclusão derivada: os dois passam de 15 FPS na CPU de teste, logo a taxa não é o gargalo e o critério decisivo passa a ser memória e energia, em que o **YOLOv4-tiny** é 2,9× menor em disco e tem 2,8× menos parâmetros. Trade-off reconhecido: o SSD foi 1,34× mais rápido (54,0 contra 40,3 FPS); se a latência for a restrição dominante (veículo rápido ou CPU embarcada em que o YOLO caia abaixo de 15 FPS), a escolha se inverte.

Rastreamento (Item B, YOLOv4-tiny + IoU)	Valor
Entradas cumulativas / saídas cumulativas	53 / 44

Rastreamento (Item B, YOLOv4-tiny + IoU)	Valor
ID switches estimados (heurística)	6 em 79,5 s de vídeo = 4,53 por minuto
Custo do rastreador	≈ 0,07 ms por frame (0,063 a 0,066 ms nas execuções)
Parâmetros	IoU ≥ 0,30; confirmação após 2 detecções; encerramento após 15 frames perdidos; trilha de 30 frames

5.3 Evidências

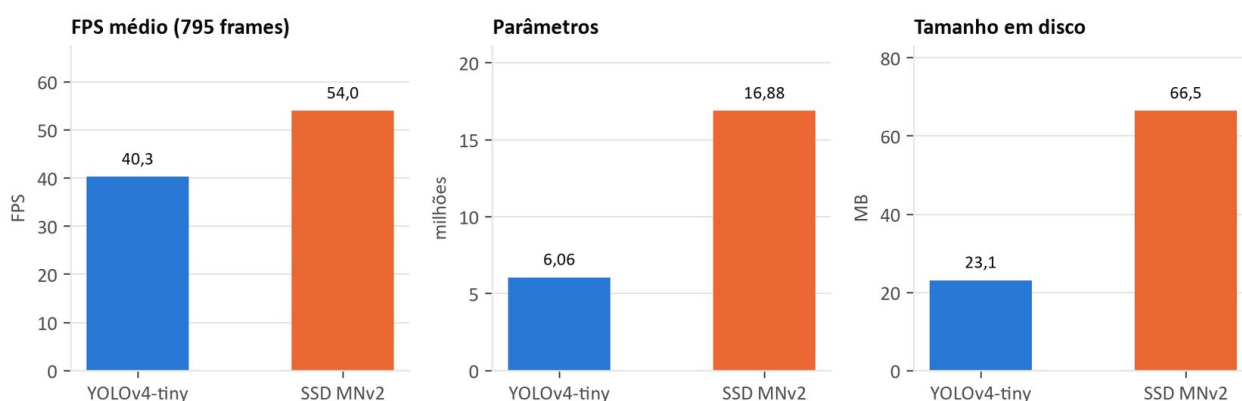


Figura 5.1 – Item A: FPS, parâmetros e tamanho em disco dos dois detectores, medidos na mesma execução.

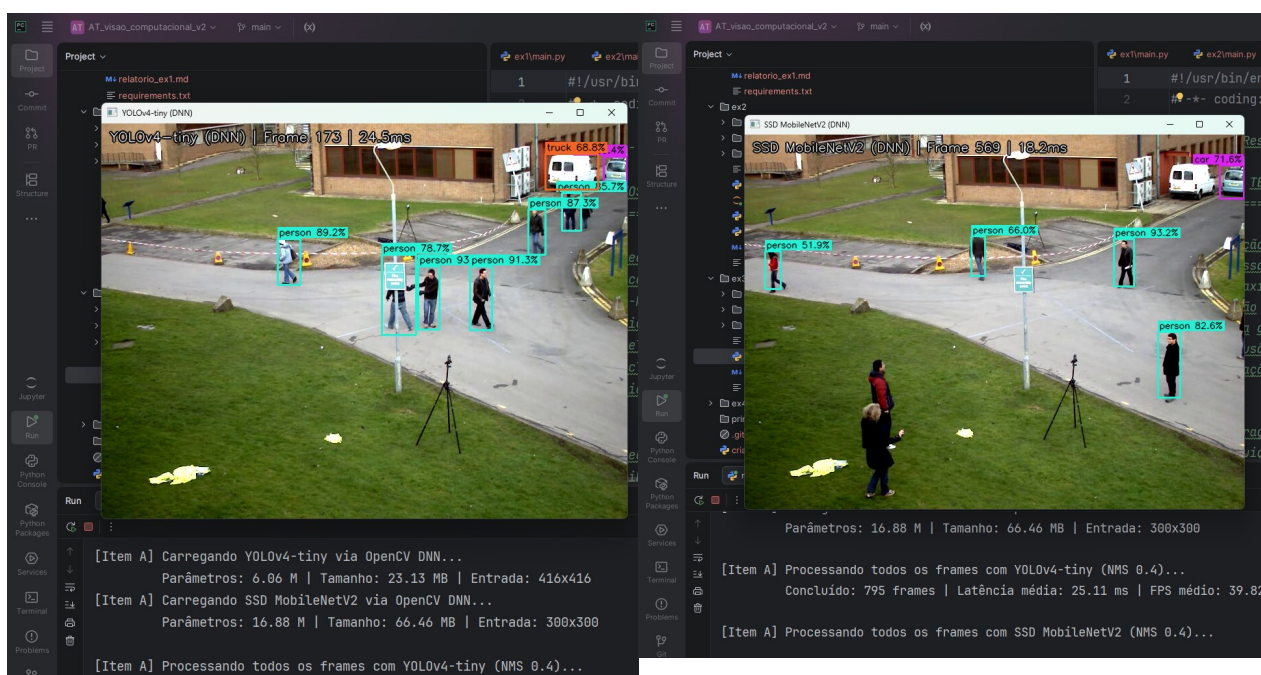


Figura 5.2 – Prints das janelas ao vivo: YOLOv4-tiny (frame 173, 24,5 ms) e SSD MobileNetV2 (frame 569, 18,2 ms), com caixas, rótulos e confiança; nos terminais, parâmetros e tamanho de cada modelo e a conclusão do YOLO (795 frames, 25,11 ms, 39,82 FPS com janela aberta).



Figura 5.3 – Item A: frames gravados em saídas/ pelos dois detectores (item_a_yolo_frame.png e item_a_ssd_frame.png).



Figura 5.4 – Item B: frames 60 e 120 de saídas/item_b_rastreamento.avi (vídeo de demonstração) com ID persistente, trilha colorida dos últimos 30 frames e HUD de objetos presentes, entradas, saídas e ID switches estimados.

Vídeo de demonstração do Exercício 3b: ex3/saidas/item_b_rastreamento.avi (795 frames, 768x576, XVID, ≈ 22 MB), gerado pela execução de referência. Os vídeos item_a_yolo.avi e item_a_ssd.avi (≈ 21 MB cada) mostram os feeds do Item A. Os três são reproduzíveis com o comando da seção 2.3.

5.4 Síntese da discussão ética (relatorio_drone_etica.md)

- **Aplicação:** contagem agregada de pedestres para gestão de eventos, mobilidade e planejamento urbano; desafios próprios da visada aérea (egomotion do drone, pedestres com menos de 30x30 px, limite térmico e energético a bordo).
- **Privacidade por design:** inferência e rastreamento na borda, sem transmitir nem gravar o vídeo bruto; descarte dos frames após gerar metadados agregados; nenhum reconhecimento facial ou identificação civil.
- **Riscos:** vigilância em massa e efeito inibidor sobre reunião pacífica; vieses de modelos treinados em visada horizontal (mais falsos negativos em tomadas zenitais, sombras, oclusão por árvores); decisões automáticas exigem supervisão humana, sobretudo porque a métrica de ID switch é estimada.
- **Transparência:** sinalização visível, aviso público de finalidade e órgão responsável, e conformidade com LGPD/GDPR.

6. Exercício 4: segmentação semântica e relatório integrativo

6.1 Decisões adotadas e justificativas

- **DeepLabV3-MobileNetV2 treinado no ADE20K (150 classes)**, grafo congelado (.pb) do model zoo do DeepLab carregado com tf.compat.v1, sem download em tempo de execução. O ADE20K foi escolhido porque tem as classes de cena urbana que o enunciado cita (road, sidewalk, person, car, building, grass), ao contrário do PASCAL VOC.
- **Seis cenas externas** (mínimo cinco): parque, jardim, playground e uma rua com calçada, van e pedestres (img06.png, frame 60 de vtest.avi). O código processa todas as imagens de entradas/.
- **Três saídas por cena**: mapa de classes colorido (mapa_*), sobreposição semitransparente com alfa 0,5 (overlay_*) e painel Original | DeepLabV3 | HSV com legenda de cores e porcentagens (painel_comparativo_*). A porcentagem de área por classe é impressa no terminal e gravada em porcentagens_classes.csv.
- **HSV com as mesmas regras do TP1** (azul = céu/água, H 90–130; verde = vegetação, H 30–90; vermelho = matiz vermelho; cinza = restante), para que a comparação seja com a técnica realmente estudada, não com um HSV ajustado.
- **Aquecimento antes de cronometrar** a rede, para que a latência reportada (58 a 73 ms por imagem, lado máximo 513 px) não inclua a inicialização do grafo.
- **Discussão no código**: DeepLab entrega semântica (separa pista de calçada, pessoa de poste) ao custo de dezenas de milissegundos e de dependência do domínio de treino; HSV custa < 3 ms mas confunde céu, água e sombra e não distingue asfalto de concreto, o que o torna inaceitável como única fonte para um veículo.
- **Relatório integrativo gerado pelo código com números medidos**. A função gerar_relatorio_integrativo escreve o Markdown com os valores dos Exercícios 1 a 3 e os do Ex4 medidos na própria execução, e valida de fato as cinco seções obrigatórias (por título) e a contagem de palavras. O que não foi medido (mAP, mIoU, ID switches reais) está escrito como não medido, sem valores de literatura.

6.2 Resultados medidos

Métrica (Item A)	Valor
Latência DeepLabV3 por imagem (após aquecimento)	58 a 73 ms (lado máximo 513 px); 2,34 M parâmetros
Latência HSV (3 faixas)	2,8 a 2,9 ms
Cena de rua img06.png: classes principais	grass 50,7 %, road 23,2 %, building 11,3 %, sidewalk 4,4 %, person 3,7 %, wall 1,3 %, van 0,7 %, car 0,5 %
Cena de parque img01.png: classes principais	sky 41,6 %, grass 27,0 %, tree 16,7 %, person 10,2 %, water 1,4 %, house 1,2 %
Validação do relatório integrativo (Item B)	Resultado
Arquivo	ex4/relatorio_integrativo.md (cópia idêntica em ex4/saidas/)
Palavras	2328 no total; 975 de texto puro, sem tabelas, títulos e blocos de código (mínimo exigido: 800)
Seções obrigatórias	5 / 5 encontradas por título: diagrama, tabela comparativa, análise 5 W, arquitetura proposta, 3 lacunas
Técnicas com dados reais	12 linhas na tabela comparativa, todas com latência medida neste AT
Veredito impresso pelo exercicio04b.py	VALIDADO

6.3 Evidências do Item A

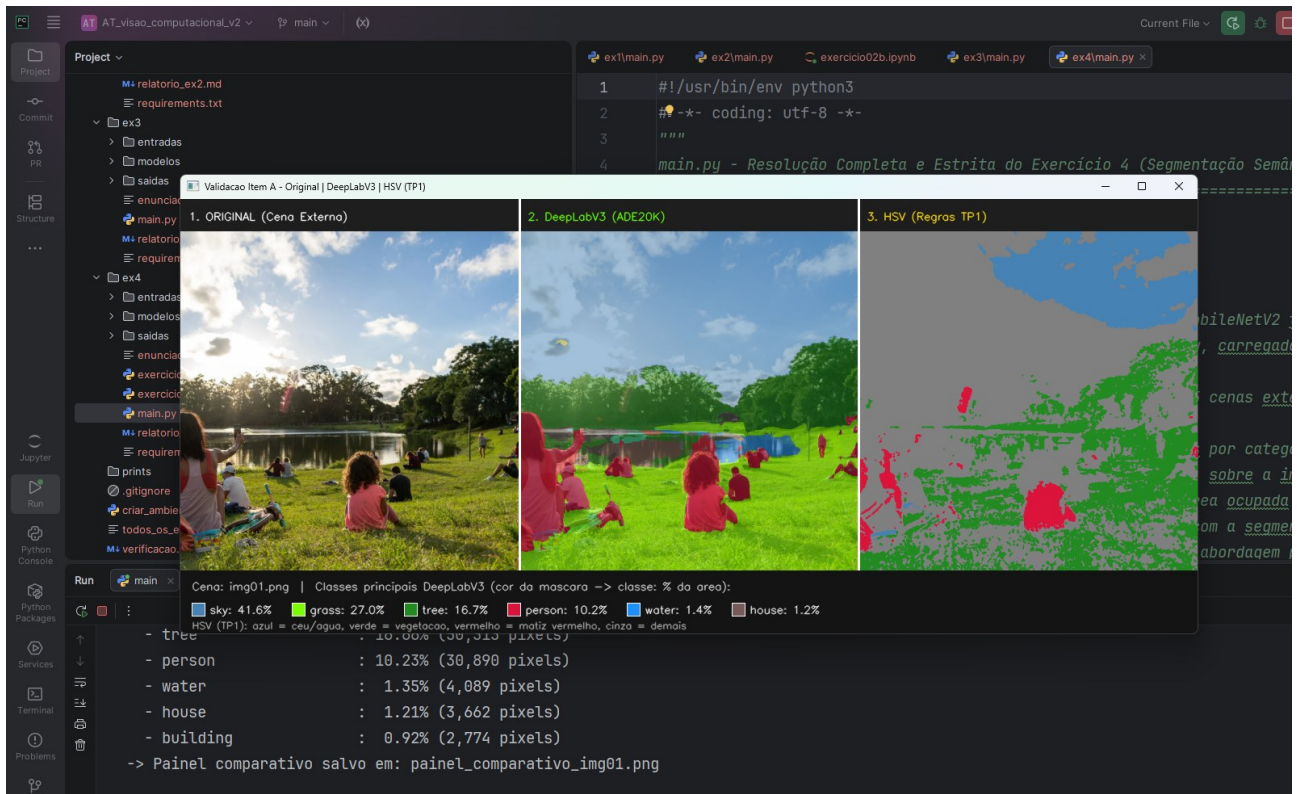


Figura 6.1 – Print: janela "Validacao Item A" com o painel Original | DeepLabV3 (ADE20K) | HSV (TP1) da cena img01.png, legenda de cores por classe e, no terminal, a porcentagem de área por classe.



Figura 6.2 – Painel gravado para a cena de rua img06.png: o DeepLab separa pista (road), calçada (sidewalk), pessoas e van; o HSV por regras só distingue vegetação, azul e vermelho.



Figura 6.3 – Sobreposição semitransparente (alfa 0,5) do DeepLabV3 em três das outras cenas (saidas/overlay_deeplab_img02/03/05.png).

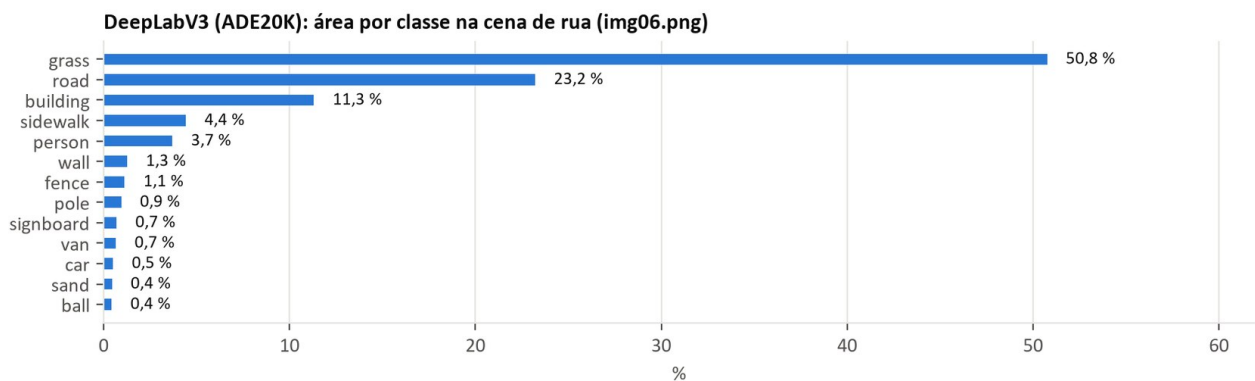


Figura 6.4 – Área por classe na cena de rua, a partir de saidas/porcentagens_classes.csv (classes com $\geq 0,4$ %).

6.4 Síntese do relatório integrativo (Item B)

O relatório integrativo completo está em ex4/relatorio_integrativo.md. Abaixo, o essencial de cada seção obrigatória.

Diagrama do pipeline completo



Figura 6.5 – Pipeline implementado ao longo da disciplina, com a latência medida de cada bloco (no Markdown o diagrama está em Mermaid, com um sétimo bloco de fusão e decisão).

Tabela comparativa das técnicas com métricas reais (CPU i9-14900KF, sem GPU)

Técnica	Origem	Latência	FPS	Qualidade medida	Memória
Calibração (K, dist)	Ex1 A	< 1 s / 20 vistas	–	MRE 0,048 px; fx, fy < 0,05 % de erro	< 10 MB
solvePnP + RA	Ex1 B	17,6 ms	56,8	reproj 0,04–0,06 px; jitter 0,20 px/frame ²	≈ 15 MB
Undistort 1280×720	Ex1 A / Ex2 B	7,3 ms	137	retificação visual	≈ 25 MB

Técnica	Origem	Latência	FPS	Qualidade medida	Memória
Segmentação HSV	TP1	1,8–2,9 ms	200–550	sem semântica	< 15 MB
ORB 2000 pontos	TP2	81,4 ms	12	62 keypoints na ROI	≈ 30 MB
HOG+SVM 1280×720	TP3	77,7 ms	13	0 det. (sem pessoas); P/R não medidos	≈ 45 MB
MobileNetV2 (DNN)	Ex2 A	5,6 ms	178	top-1 90 %	94 MB RSS
MobileNetV2 (Keras)	Ex2 A	64,7 ms	15	top-1 90 %	464 MB RSS
YOLOv4-tiny 416	Ex3 A	24,8 ms	40,3	5659 det.; mAP não medido	23,1 MB disco
SSD MobileNetV2 300	Ex3 A	18,5 ms	54,0	4942 det.; mAP não medido	66,5 MB disco
Rastreador IoU	Ex3 B	0,07 ms	> 10 000	53 entradas, 44 saídas; 4,5 IDsw/min (estim.)	< 5 MB
DeepLabV3-MNV2 513	Ex4 A	73 ms	13,8	road 23 %, sidewalk 4 % na rua; mIoU não medido	≈ 150 MB

Viabilidade em hardware embarcado de 5 W

Hipótese de planejamento: uma placa de 5 W (Raspberry Pi 4/5, Jetson Nano em modo 5 W) entrega de 6 a 10 vezes menos desempenho por núcleo que a CPU de teste; adota-se o fator conservador de 8× sobre as latências medidas, a ser remediado no hardware alvo. Vereditos: undistort, HSV, rastreador e classificador sob demanda são viáveis; SSD e YOLO ficam limítrofes (5 a 7 FPS projetados, exigindo INT8 ou NPU); ORB com 2000 pontos, HOG+SVM contínuo e DeepLab por frame são inviáveis, devendo rodar em ROI, ser substituídos ou executados a 1–2 Hz.

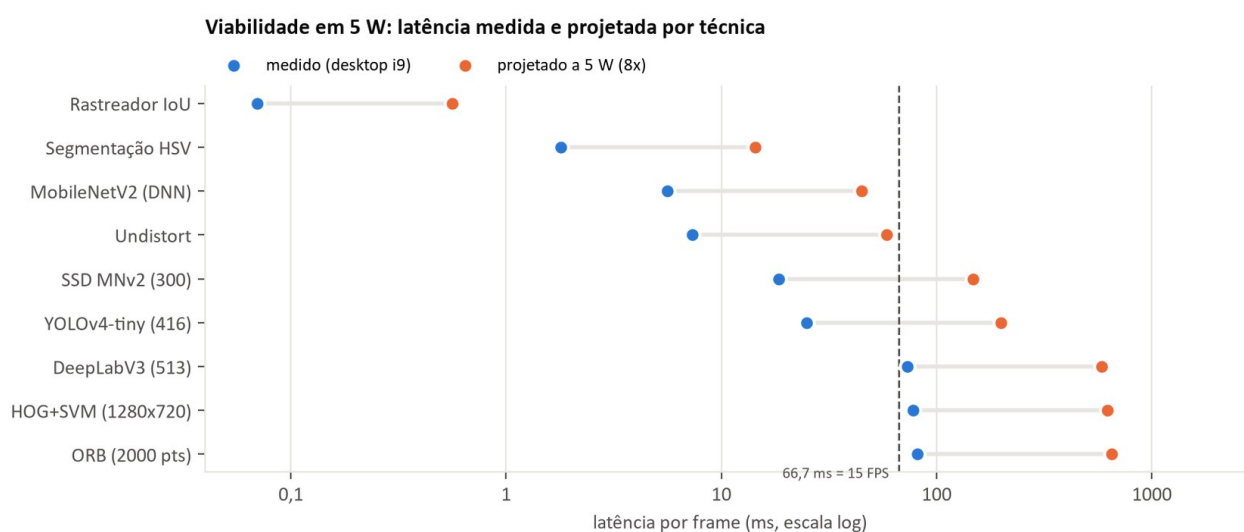


Figura 6.6 – Latência medida no desktop e projetada a 5 W (fator 8×) por técnica; a linha tracejada marca 66,7 ms, equivalente a 15 FPS.

Arquitetura proposta e lacunas para a DR4

- **Arquitetura multi-taxa com 6 técnicas:** retificação (30 Hz, CPU) → filtro HSV para semáforos e cones (30 Hz) e ORB com 300–500 pontos para odometria visual (30 Hz) → detector SSD/YOLO (10 Hz, NPU) → rastreador IoU (30 Hz, preenchendo os frames entre detecções) → DeepLabV3 a 2 Hz para área navegável → fusão em grade de ocupação → planejador local e controle.
- **Lacuna 1:** escala absoluta e profundidade com câmera monocular (no Ex1 a pose só é métrica porque o quadrado de 25 mm é conhecido). Endereçamento: LiDAR ou estéreo e calibração extrínseca

câmera-LiDAR.

- **Lacuna 2:** robustez a iluminação e clima (HSV confunde céu, água e sombra; DeepLab avaliado só em cenas diurnas). Endereçamento: radar, câmera térmica e conjuntos de dados com condições adversas.
- **Lacuna 3:** rastreamento sob oclusão e métrica de ID switch sem gabarito. Endereçamento: reidentificação por aparência (DeepSORT/ByteTrack), filtro de Kalman e avaliação MOT com IDF1/MOTA em sequências anotadas.

7. Limitações declaradas e o que não foi medido

- **Ex1:** as entradas sintéticas (imagens, vídeo e verdade.npz) não têm o script gerador no repositório; a calibração e a RA são reproduzíveis a partir dos arquivos entregues.
- **Ex2 B:** o undistort aplica a calibração da câmera virtual do Ex1 a um vídeo gravado por outra câmera. O passo é executado e cronometrado por exigência do enunciado; a limitação está no código, no notebook e no relatório.
- **Ex3:** não há gabarito anotado para vtest.avi, logo mAP dos detectores e ID switches reais não são mensuráveis; a taxa de 4,53 ID switches por minuto é uma estimativa heurística que tende a subestimar o valor real.
- **Ex4:** mIoU não medido pelo mesmo motivo; o diagrama do pipeline no Markdown está em Mermaid (renderiza no GitHub e no VS Code), e neste PDF foi redesenhado como figura.
- **Projeção para 5 W:** o fator 8× é hipótese de planejamento; a decisão final entre SSD e YOLO e a taxa da segmentação devem ser tomadas com medições no hardware alvo, com o consumo lido no barramento de alimentação.
- **Prints:** feitos com as janelas abertas, o que explica pequenas diferenças de FPS em relação às tabelas.

8. Índice das evidências de execução

Capturas de tela da pasta prints/ (nome original do arquivo), na ordem em que aparecem neste relatório:

Arquivo em prints/	Exercício / item	O que mostra	Figura
Captura de tela 2026-09-23 221450.png	Ex1 B	video_aumentado.avi em reprodução: cubo 3D, eixos, HUD rvec/tvec, legenda de faces; terminal com exit code 0	3.4
Captura de tela 2026-09-23 221604.png	Ex1 B	Terminal: resumo da validação (150/150 frames, jitter 0,1975, veredito ESTÁVEL)	3.5
Captura de tela 2026-09-23 221642.png	Ex2 A	Janela OpenCV com top-3 (gas_pump 86,6 %) e terminal com o top-1 das 10 imagens	4.2
Captura de tela 2026-09-23 221804.png	Ex2 B	Frame do pipeline integrado com HUD de tempo por etapa (179,18 ms, 5,6 FPS)	4.5
Captura de tela 2026-09-23 222014.png	Ex2 B	Notebook exercicio02b.ipynb executado no PyCharm, figura da célula final e terminal de conclusão (28,96 s)	4.6
Captura de tela 2026-09-23 222055.png	Ex3 A	Janela YOLOv4-tiny (frame 173, 24,5 ms) e terminal com parâmetros e tamanho dos modelos	5.2
Captura de tela 2026-09-23 222128.png	Ex3 A	Janela SSD MobileNetV2 (frame 569, 18,2 ms) e terminal com a conclusão do YOLO (795 frames)	5.2
Captura de tela 2026-09-23 222244.png	Ex4 A	Janela com o painel Original DeepLabV3 HSV (img01) e terminal com porcentagens por classe	6.1

Arquivos gerados pelos scripts e usados como evidência:

Arquivo	Exercício / item	Figura
ex1/saidas/poses_estimadas.csv (gráfico gerado para este relatório)	Ex1 B	3.1
ex1/saidas/painel_undistort.png	Ex1 A	3.2
ex1/saidas/frame_cubo_inicial.png, frame_cubo_meio.png, frame_cubo_final.png	Ex1 B	3.3
ex2/saidas/top3/*.png (10 imagens)	Ex2 A	4.3

Arquivo	Exercício / item	Figura
ex2/saidas/frame_pipeline_integrado.png	Ex2 B	4.5
ex3/saidas/item_a_yolo_frame.png, item_a_ssd_frame.png	Ex3 A	5.3
ex3/saidas/item_b_rastreamento_frame_0060.png, _0120.png	Ex3 B	5.4
ex3/saidas/item_b_rastreamento.avi (vídeo de demonstração), item_a_yolo.avi, item_a_ssd.avi	Ex3 A e B	–
ex4/saidas/painel_comparativo_img06.png; overlay_deeplab_img02/03/05.png	Ex4 A	6.2, 6.3
ex4/saidas/porcentagens_classes.csv (gráfico gerado para este relatório)	Ex4 A	6.4
ex4/relatorio_integrativo.md, ex3/relatorio_drone_etica.md, ex1/relatorio_ex1.md, ex2/relatorio_ex2.md	Relatórios por exercício	–